The Most Efficient Protocol for PAKE:
What Exact Stuff Do You Need to Hash at the End?

Jiayu Xu

Oregon State University, xujiay@oregonstate.edu

1 Introduction

2 Preliminaries

3 Security Analysis of the "Hash Everything" Version

3.1 The Simulator

3.1.1 Simulation Strategy

A large portion of the simulator is routine; below we explain the idea behind the non-trivial parts.

Simulating honest P-to-P' message. When P's session begins, $\mathcal{S}$ must simulate its message (s, T) to P'. This can be easily done: just sample (s, T) at random. However, every time $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$ (let the result be t^*), $\mathcal{S}$ needs to generate a KEM key pair (pk^*, sk^*) for later use (see the "Extracting password guess from malicious P'" paragraph below) and make sure that (pk^*, sk^*) is consistent with the RO queries. This can be done as follows: $\mathcal{S}$ computes

$$r^* := s \oplus t^*, \quad R^* := T/pk^*,$$

and programs $H_1(\mathsf{pw}^*, r^*) := R^*$. This also allows $\mathcal{S}$ to store the association between pw^* and (pk^*, sk^*). (Note that (pk^*, sk^*) needs to be freshly generated every time $\mathcal{A}$ queries $H_2(\star, T)$; otherwise R^* never changes and $\mathcal{A}$ can easily find collisions in H_1.)

Extracting password guess from malicious P. When the adversary $\mathcal{A}$ sends a message (s^*, T^*) to P', if it is different from the honest message (s, T), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding

TestPwd command to $\mathcal{F}_{\text{PAKE}}$. There are two types of attack, which need to be addressed separately:

1. *"Normal" online attack:* $\mathcal{A}$ executes P's algorithm on a password guess pw^*. That is, $\mathcal{A}$ samples public key pk^* and randomness r^*, computes

$$R^* := H_1(\mathsf{pw}^*, r^*), \qquad T^* := pk^* \cdot R^*,$$
$$t^* := H_2(\mathsf{pw}^*, T^*), \qquad s^* := t^* \oplus r^*,$$

   and sends (s^*, T^*) to P'.
   It takes a while to realize how to extract pw^* from (s^*, T^*). First, $\mathcal{S}$ should scan all $H_2(\star, T^*)$ queries, which yields multiple pw^* values (and multiple t^* values); $\mathcal{S}$ must decide which one is the "actual" password guess. This can be done via the following. For each such (pw^*, t^*) pair, $\mathcal{S}$ computes the corresponding $r^* := s^* \oplus t^*$, and checks if (and when) $H_1(\mathsf{pw}^*, r^*)$ was queried. *With overwhelming probability, there is at most one such H_1 query made before the $H_2(\mathsf{pw}^*, T^*)$ query*, from which $\mathcal{S}$ can extract the "actual" password guess pw^*. (This special $H_1(\mathsf{pw}^*, r^*)$ query is called the "anchor query" in [MRR20, JRX25].)

2. *Related key attack:* This attack works only after $\mathcal{A}$ receive an honest (s, T) pair from P. $\mathcal{A}$ skips sampling pk^* or r^*; instead, $\mathcal{A}$ picks any $\Delta \in \mathbb{G}$, computes $T^* := T \cdot \Delta$ and

$$t^* := H_2(\mathsf{pw}^*, T), \qquad (t^*)' := H_2(\mathsf{pw}^*, T^*),$$
$$\delta := t^* \oplus (t')^*, \qquad s^* := s \oplus \delta,$$

   and sends (s^*, T^*) to P'.
   Let us first explain what $\mathcal{A}$ is trying to achieve here. The same randomness r yields two valid messages, (s, T) and (s^*, T^*), such that

$$s \oplus s^* = H_2(\mathsf{pw}, T) \oplus H_2(\mathsf{pw}, T^*),$$

   which allows $\mathcal{A}$ to pick T^* and "reverse engineer" the corresponding s^* if it guesses pw correctly. (s^*, T^*) implicitly encrypts $pk^* = T^*/H_1(\mathsf{pw}, r)$, which $\mathcal{A}$ can compute as $pk \cdot (T^*/T)$.
   To extract pw^* from (s^*, T^*), $\mathcal{S}$ should scan all $H_2(\star, T)$ and $H_2(\star, T^*)$ queries (which yield multiple candidate pw^* values) and check which ones satisfy

$$H_2(\mathsf{pw}^*, T) \oplus H_2(\mathsf{pw}^*, T^*) = s \oplus s^*.$$

   With overwhelming probability, at most one pw^* will satisfy this equation.

Extracting password guess from malicious P'. When the adversary $\mathcal{A}$ sends a message (c^*, τ^*) to P, if $\mathcal{A}$ is not passive (i.e., it is not the case that $(s^*, T^*) = (s, T)$ *and* $(c^*, \tau^*) = (c, \tau)$), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding TestPwd command to $\mathcal{F}_{\text{PAKE}}$.

$\mathcal{S}$ should find the H_{tag} query whose result is τ^*, which includes $(\mathsf{pw}^*, pk^*, K^*)$. However, pw^* constitutes a password guess only if it is consistent with pk^* and K^*, i.e., $K^* = \mathsf{Decap}(sk^*, c^*)$ for sk^* corresponding to pk^*. $\mathcal{S}$ can check this condition because it has stored the correspondence between pw^* and (pk^*, sk^*) (see the "Simulating honest P-to-P' message" paragraph above), and can use sk^* to check if the equation holds. (This extraction strategy does not work anymore if *any* of pw^*, pk^*, c^* is not included in H_{tag}. In later sections we will explain how to simulate other versions of the protocol.)

3.1.2 Formal Description of the Simulator

3.2 Hybrid Argument

We now prove that for any PPT environment $\mathcal{Z}$, the simulator $\mathcal{S}$ in Section 3.1.2 generates an ideal-world view indistinguishable from $\mathcal{Z}$'s real-world view. The sequence of games start from the real world (Game 0) and ends at the ideal world.

Game 0: This is the real world. Recall that the passwords of P and P' are pw and pw', respectively; the flow of events is:

- P sends (s, T) to P';
- It is intercepted by the man-in-the-middle adversary $\mathcal{A}$, who sends (s^*, T^*) (which may or may not be equal to (s, T)) to P';
- P' outputs session key SK' and sends (c, τ') to P;
- It is again intercepted by $\mathcal{A}$, who sends (c^*, τ^*) to P;
- P computes τ. If $\tau^* = \tau$ then P outputs SK, otherwise P outputs $\perp$.

(Note that there are three τ values: τ' computed and sent by P', τ^* sent by $\mathcal{A}$, and τ computed by P used in P's check.)

$\mathcal{A}$ is passive, except maybe modifying the tag.

Game 1: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, do the following:

- If $\tau^* = \tau'$ (i.e., $\mathcal{A}$ is passive), set $SK := SK'$.
- If $\tau^* \neq \tau'$, set $SK := \perp$.

By correctness of the protocol (see Section 2), in Game 0 $\tau = \tau'$ except with probability $\mathbf{Adv}^{\text{correctness}}$. This means that except with probability $\mathbf{Adv}^{\text{correctness}}$, P's check $\tau^* \stackrel{?}{=} \tau$ in Game 0 is equivalent to P's check $\tau^* \stackrel{?}{=} \tau'$ in Game 1. If the check passes then both games set $SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c, K)$, otherwise both games set $SK := \perp$. We have

$$\mathbf{Dist}^{0,1}_{\mathcal{Z}} = \mathbf{Adv}^{\text{correctness}}.$$

This game corresponds to games G_3–G_5 in [ABJ25]. We believe having a single game that handles both the $\tau^* = \tau'$ case and the $\tau^* \neq \tau'$ case is much cleaner.

P and P''s passwords match, and $\mathcal{A}$ forwards the P-to-P' message.

Game 2: In the case where $\tau^* \neq \tau'$, set $SK := \perp$ if either of the following happens:

- $\mathcal{A}$ never makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ query resulting in τ^*, or
- $\mathcal{A}$ makes two different $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ queries, both resulting in τ^*.

In the first event, Game 1 and Game 2 differ if $\mathcal{A}$ never makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ query resulting in τ^*, yet P's check $\tau^* \stackrel{?}{=} H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ passes. Clearly,

- $\mathcal{A}$ does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ (otherwise the result is τ^*), and
- P' does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ (otherwise $\tau' = H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K) = \tau^*$).

So $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ is independent of $\mathcal{A}$'s view, and the probability that $\mathcal{A}$ sends $\tau^* = H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ is $1/2^n$. The second event is collision in H_{tag}, whose probability is upper-bounded by $q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1)/2^{n+1}$. Overall,

$$\mathbf{Dist}^{1,2}_{\mathcal{Z}} \leq \frac{q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1) + 2}{2^{n+1}}.$$

This game corresponds to games G_6 and G_7 in [ABJ25].

Game 3: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, sample $SK', \tau' \leftarrow \{0,1\}^n$. ($c$ is still computed as in Game 2.)

Let Query_1 be the event that $\mathcal{A}$ queries *either* $H_{\mathsf{key}}(\mathsf{pw}, pk, c, K')$ *or* $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, K')$. Clearly Game 2 and Game 3 are identical unless Query_1 happens. We upper-bound $\Pr[\mathsf{Query}_1]$ via a reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property of KEM:

Reduction $\mathcal{R}_{\text{OW-1PCA1}}$: // Boxed text indicates the single PC oracle query; same for reductions below

Sample $i \leftarrow [q_{\mathsf{key}} + q_{\mathsf{tag}}]$ as a guess that $\mathcal{A}$'s i-th H_{key} or H_{tag} query triggers Query_1.
On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T). (Note that computing (s, T) does not require knowing sk.) Send (s, T) to $\mathcal{A}$.
2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{OW-PCA1}}$'s inputs.)

3. On (c^*, τ^*) from $\mathcal{A}$,
   (a) If $c^* = c$, do what's described in Game 1.
   (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)
   (c) Query $\mathsf{PC}(sk, c^*, K^*)$.
   If the result is 1, output $SK := H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ to $\mathcal{Z}$.
   If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.
4. On $\mathcal{A}$'s i-th H_{key} or H_{tag} query, if it is in the form of $H_{\mathsf{key}}(\mathsf{pw}, pk, c, (K')^*)$ or $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, (K')^*)$ for some $(K')^*$, output $(K')^*$; otherwise abort.

Assuming Query_1 happens, the only difference between Game 2/3 and $\mathcal{R}_{\text{OW-1PCA1}}$'s simulation (until Query_1 happens) is that $\mathcal{R}_{\text{OW-1PCA1}}$ uses its input pk on the P side, and then uses its input $c \leftarrow \mathsf{Encap}(pk)$ on the P' side. Since we assume $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, in Game 2/3 P''s algorithm will recover $pk' = pk$, so there c is also generated as $\mathsf{Encap}(pk)$ and thus $\mathcal{R}_{\text{OW-1PCA1}}$ simulates Game 2/3 perfectly. Furthermore, if $\mathcal{R}_{\text{OW-1PCA1}}$'s guess is correct then it wins the OW-1PCA game. We have

$$\mathbf{Dist}^{2,3}_{\mathcal{Z}} \leq \Pr[\mathsf{Query}_1] \leq (q_{\mathsf{key}} + q_{\mathsf{tag}})\mathbf{Adv}^{\text{OW-1PCA}}_{\mathcal{R}_{\text{OW-1PCA1}}}.$$

This game corresponds to game G_8 in [ABJ25]. Note that if we reduce to the OW-PCA property of KEM where the adversary can query the PC oracle polynomially many times, then step 4 of the reduction above can query $\mathsf{PC}(sk, c, (K')^*)$ $q_{\mathsf{key}} + q_{\mathsf{tag}}$ times to check which of $\mathcal{A}$'s query (if any) triggers Query_1, hence avoiding the $q_{\mathsf{key}} + q_{\mathsf{tag}}$ loss in its advantage. This is the approach taken by [ABJ25].

Game 4: Again in the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, generate P''s KEM public key $pk' \leftarrow \mathsf{Gen}$ and compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$. (Note that K' is not used anywhere since Game 3 — in particular, P' does not use K' to compute SK' or τ' anymore — so we do not need to explicitly mention it.)

The difference between Game 3 and Game 4 is that in Game 3 both P and P' use the same pk, whereas in Game 4 P uses pk and P' uses an independent pk'. We construct a reduction $\mathcal{R}_{\text{ANO-1PCA}}$ to the ANO-1PCA property of KEM. This reduction, in fact, is very similar to the reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property in Game 3, and we highlight their differences in blue:

Reduction $\mathcal{R}_{\text{ANO-1PCA}}$:

On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk rather than $(pk, \star) \leftarrow \mathsf{Gen}$. (Note that computing (s, T) does not require knowing sk.) Send (s, T) to $\mathcal{A}$.

2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}',$ "respondent") from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{ANO-PCA1}}$'s inputs.)

3. On (c^*, τ^*) from $\mathcal{A}$,

   (a) If $c^* = c$, do what's described in Game 1.

   (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

   (c) $\boxed{\text{Query } \mathsf{PC}(sk, c^*, K^*)}$. // This is a valid query since we do this step only if $c^* \neq c$
   If the result is 1, output $SK := H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ to $\mathcal{Z}$.
   If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. When $\mathcal{Z}$ outputs bit b^*, copy $\mathcal{Z}$'s output.

$\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk on the P side and c on the P' side. If $b = 0$ then $c \leftarrow \mathsf{Encap}(pk)$, i.e., both P and P' use the same pk, so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 3; if $b = 1$ then $c \leftarrow \mathsf{Encap}(pk')$, i.e., P' uses an independently generated pk', so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 4. We have

$$\mathbf{Dist}^{3,4}_{\mathcal{Z}} \leq \mathbf{Adv}^{\text{ANO-1PCA}}_{\mathcal{R}_{\text{ANO-1PCA}}}.$$

This game corresponds to game G_9 in [ABJ25]. However, there are two differences:

1. *[ABJ25] reduces to ANO-PCA (where the KEM adversary can query its PCA oracle polynomially many times), rather than ANO-PCA1. Unlike in Game 3, here reducing to this stronger assumption does not provide any advantage.*

2. *The assumption [ABJ25] reduces to is even further stronger in that their KEM adversary's inputs are (pk, pk', c), whereas we do not give pk' to the KEM adversary. Note that the reduction does not need pk' at all (it only needs to send c which is an encapsulation of either pk or pk'), so the assumption in [ABJ25] is unnecessarily strong.*

References

[ABJ25] Afonso Arriaga, Manuel Barbosa, and Stanislaw Jarecki. NoIC: PAKE from KEM without ideal ciphers. Cryptology ePrint Archive, Report 2025/231, 2025.

[JRX25] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? In *EUROCRYPT 2025*, pages 451–481, 2025.

[MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In *CCS 2020*, pages 425–442, 2020.